

Diplomado en Programación

Proyecto: Sistema de Gestión Académica

PARTE I: El Escenario (Contexto Corporativo y Problemática)

1. Antecedentes de la Empresa

DiplomadosOnline.com es una plataforma educativa líder en la región que ofrece formación especializada en tecnología y negocios. En sus inicios, la academia manejaba grupos pequeños de 50 estudiantes por cohorte. Sin embargo, debido al éxito de sus programas, la matrícula ha experimentado un crecimiento exponencial, gestionando actualmente miles de estudiantes activos e históricos y un gran cuerpo docente en múltiples países.

2. La Crisis Operativa

A pesar de ser una empresa tecnológica, el departamento de *Control de Estudios* sigue utilizando herramientas ofimáticas tradicionales. Todo el registro de la academia recae en un gigantesco archivo de Excel compartido llamado **Alumnos_y_Profesores_General.xlsx**. Este manejo manual ha desencadenado una severa crisis operativa con tres puntos críticos:

- **Falta de Integridad de Datos:** Es común encontrar estudiantes y profesores con números de cédula/ID duplicados, nombres mal escritos o asignados a programas incorrectos debido a errores de tipeo humano.
- **Falta de Persistencia Confiable:** Al depender de un solo archivo Excel, si este se corrompe o se cierra inesperadamente, se pierden horas de trabajo. No existe un sistema de base o de archivos de texto plano bien estructurado.
- **Cuello de Botella en la Certificación:** Al finalizar un periodo académico, saber quiénes se gradúan es un caos. Un empleado debe revisar fila por fila, calcular el promedio manualmente según las reglas de cada programa y luego copiar los nombres de los aprobados para enviarlos al *Departamento de Certificados*. Este proceso retrasa la entrega de diplomas hasta por varias semanas.

3. El Reto Técnico y la Prueba de Concepto

La junta directiva de **DiplomadosOnline** ha decidido desarrollar un **Sistema de Gestión Académica (SGA-DO)** propio. Tu misión como desarrollador es construir el **Motor Core (Backend)** de la aplicación por consola, asegurando que los datos se guarden en archivos de texto (**.txt**) para que persistan en el disco duro.

Para evaluar la versatilidad de las tecnologías, deberás construir **la misma solución** en tres lenguajes distintos a lo largo del diplomado: **Python, Java y C++**, adaptando las estructuras a la filosofía de cada entorno.

PARTE II: Arquitectura de Clases y Reglas de Negocio

Para garantizar un diseño de software limpio y escalable, el sistema debe estructurarse utilizando los conceptos de **Herencia y Polimorfismo** bajo el siguiente esquema de clases:

1. Jerarquía de Personas

- **Persona**: Contiene los atributos comunes: **Cédula/ID**, **Nombre Completo** y **Correo Electrónico**.
- **Alumno**: Atributos adicionales: **Lista de Notas** (máximo 3 notas por programa) y el **Programa Académico** asignado.
- **Profesor**: Atributos adicionales: **Especialidad Académica** (ej. Python, Java, C++) y **Materia Asignada**.

2. Jerarquía de Programas Académicos

Existirá una clase base llamada **ProgramaAcademico** de la cual heredarán tres modalidades. Cada una tiene su propia regla polimórfica para calcular la aprobación:

- **Curso**: Se aprueba si el promedio de las 3 notas es mayor o igual a **10/20**.
- **Diplomado**: Se aprueba si el promedio de las 3 notas es mayor o igual a **14/20**.
- **Bootcamp**: No se aprueba por promedio. Exige estrictamente que **ninguna nota individual sea menor a 14/20** (si tiene una sola nota por debajo de 14, está reprobado).

PARTE III: Requerimientos Funcionales y Menú de Consola

El sistema debe ejecutarse en un bucle continuo por consola y mostrar un menú interactivo. El programa no debe cerrarse hasta que el usuario seleccione explícitamente la opción de salida.

El Menú de Consola

Al iniciar la aplicación, se debe desplegar la siguiente interfaz de texto:

```
=====
SGA-DO: SISTEMA DIPLOMADOSONLINE
=====
1. Registrar Alumno
2. Registrar Profesor
3. Registrar Notas a un Alumno
4. Deshacer Último Registro de Nota
5. Generar Cola de Certificados
6. Mostrar Reporte General
7. Salir
=====
Seleccione una opción (1-7):
```

Descripción Detallada de las Opciones

1. **Registrar Alumno:** Solicita Cédula, Nombre, Correo y Tipo de Programa (*Curso, Diplomado o Bootcamp*). Guarda los datos en memoria y los escribe inmediatamente en el archivo `alumnos.txt`.
2. **Registrar Profesor:** Solicita Cédula, Nombre, Correo, Especialidad y Materia. Guarda los datos en memoria y los escribe en el archivo `profesores.txt`.
3. **Registrar Notas a un Alumno:** Solicita la Cédula del alumno. Si lo encuentra (leyendo la memoria o el archivo), permite ingresar una calificación.
4. **Deshacer Último Registro de Nota:** Funciona como un `Ctrl + Z`. Cada vez que el usuario añade una nota en la Opción 3, la acción se guarda en una **Pila**. Si el usuario comete un error de tipeo, al seleccionar esta opción, el sistema remueve la última nota ingresada de la lista del alumno utilizando la lógica LIFO (*Last In, First Out*).
5. **Generar Cola de Certificados:** El sistema analiza a todos los alumnos registrados en el archivo. Aquellos que cumplan con los criterios de aprobación (según sean de Curso, Diplomado o Bootcamp) se insertan en una **Cola (Queue)**. El sistema procesa la cola en orden FIFO (*First In, First Out*) y exporta la lista final de graduandos a un archivo físico llamado `certificados_pendientes.txt`.
6. **Mostrar Reporte General:** Lee los archivos `alumnos.txt` y `profesores.txt` e imprime en pantalla de forma ordenada la lista completa de profesores activos y alumnos registrados con sus respectivos estatus.
7. **Salir:** Guarda cualquier cambio pendiente, limpia la memoria y cierra la aplicación de forma segura.

Diccionario de Datos y Persistencia

1. Estructura del Excel `Alumnos_y_Profesores_General.xlsx`

Este archivo representa el caos manual actual de la empresa

Cédula / ID	Tipo	Nombre Completo	Correo Electrónico	Programa / Especialidad	Materia Asignada	Nota 1	Nota 2	Nota 3	Promedio (Manual)
V-111	Alumno	Ana Silva	ana@email.com	Curso	N/A	10	11	09	10.0

P-99	Profesor	Dr. Pedro Infante	pedro@email.com	Backend	C++ y Algoritmos	N/A	N/A	N/A	N/A
V-22	Alumno	Carlos Pérez	carlos@email.com	Diplomado	N/A	14	13	14	13.6
V-33	Alumno	María Gómez	maria@email.com	Bootcamp	N/A	20	20	12	17.3
V-44	Alumno	Luis Rojas	luis@email.com	Bootcamp	N/A	14	15	14	14.3

2. Estructura de los Archivos **.txt** (El Sistema SGA-DO)

Archivo 1: **alumnos.txt**

Estructura esperada por línea:

Cedula, Nombre, Correo, TipoPrograma, Nota1, Nota2, Nota3 (Si el alumno acaba de ser registrado y aún no tiene notas, se guardan como ceros).

Data de ejemplo generada por el sistema:

V-111,Ana Silva,ana@email.com,Curso,10,11,9
V-222,Carlos Perez,carlos@email.com,Diplomado,14,13,14
V-333,Maria Gomez,maria@email.com,Bootcamp,20,20,12
V-444,Luis Rojas,luis@email.com,Bootcamp,14,15,14

Archivo 2: **profesores.txt**

Estructura esperada por línea: **Cedula, Nombre, Correo, Especialidad, Materia**

Data de ejemplo generada por el sistema:

P-999,Dr. Pedro Infante,pedro@email.com,Backend,C++ y Algoritmos
P-888,Ing. Laura Soto,laura@email.com,Frontend,Javascript Core

Archivo 3: certificados_pendientes.txt

Este archivo no es una base de datos, sino un **Reporte de Salida**. Se sobrescribe o se actualiza cada vez que el usuario ejecuta la "Opción 5: Generar Cola de Certificados" en el menú.

Aquí el formato puede ser más amigable, ya que está destinado a ser impreso por el departamento de Recursos Humanos.

Data de ejemplo generada por el sistema (basada en la data anterior):

```
=====
REPORTE DE CERTIFICADOS PENDIENTES
=====
Total de graduandos en cola: 2
```

1. [V-111] Ana Silva
 - Programa: Curso
 - Promedio Final: 10.0
 - Estatus: APROBADO
2. [V-444] Luis Rojas
 - Programa: Bootcamp
 - Promedio Final: 14.3
 - Estatus: APROBADO (Cumple regla de ninguna nota < 14)

```
=====
* Fin del reporte - Generado por SGA-DO *
```

PARTE IV: Estructura de Entregables y Carpetas

Para evaluar el proyecto, el estudiante deberá construir progresivamente un repositorio en GitHub. El avance se divide en las siguientes 7 fases que corresponden a los módulos del diplomado:

- **1) Análisis del Problema y Plan de Acción**
 - *Objetivo:* Demostrar la comprensión del problema del negocio antes de programar.
 - *Entregable:* Un documento (PDF) donde el estudiante explique con sus propias palabras la crisis de DiplomadosOnline, identifique los requerimientos del MVP y redacte su plan lógico de trabajo.
- **2) Modelado Orientado a Objetos UML**
 - *Objetivo:* Diseñar el mapa arquitectónico de clases que guiará el desarrollo en todos los lenguajes.
 - *Entregable:* Imagen o PDF del Diagrama de Clases UML mostrando las herencias (**Persona** y **ProgramaAcademico**), atributos públicos/privados y métodos.
- **3) Repositorio Git y GitHub**
 - *Objetivo:* Configurar el entorno profesional de control de versiones.

- *Entregable:* URL del repositorio público [sga-diplomadosonline](#) con la estructura de carpetas inicial ([/docs](#), [/python](#), [/java](#), [/cpp](#)) y los primeros *commits* de la documentación técnica.
- **4) Implementación en Python**
 - *Objetivo:* Crear el primer prototipo funcional interactivo usando la sintaxis ágil de Python y manejo de archivos nativos.
 - *Entregable:* Código fuente en la carpeta [/python](#).
- **5) Implementación en Java**
 - *Objetivo:* Replicar el sistema bajo las reglas estrictas de Java (encapsulamiento [private](#), colecciones oficiales y manejo de excepciones en archivos).
 - *Entregable:* Código fuente en la carpeta [/java](#) utilizando colecciones para la Pila y la Cola.
- **6) Implementación en C++**
 - *Objetivo:* Llevar el sistema al entorno compilado manejando la persistencia mediante flujos de datos ([fstream](#)) y gestión manual de memoria (punteros [new](#) y [delete](#)).
 - *Entregable:* Código fuente en la carpeta [/cpp](#).
- **7) Informe Ejecutivo y Presentación Final**
 - *Objetivo:* Defensa técnica del proyecto ante el jurado evaluador.
 - *Entregable:* Presentación (PPT/PDF) con las conclusiones del proyecto y un cuadro comparativo argumentando los pros y contras de haber programado la misma solución en Python, Java y C++.

ANEXO QA: Protocolo de Pruebas y Certificación de Calidad

Todo software profesional debe pasar por rigurosas pruebas antes de salir a producción. Este anexo se divide en dos partes: las pruebas que **el estudiante** debe realizar antes de entregar su proyecto, y las pruebas técnicas que **el profesor** ejecutará para asignar la calificación final.

Data Semilla (Obligatoria para todas las pruebas)

Para que las pruebas sean válidas, el sistema debe ser cargado inicialmente con esta información base, la cual debe seguir la misma lógica:

Cédula / ID	Nombre Completo	Perfil	Programa / Esp.	Notas a Ingresar	Resultado Interno Esperado
V-101	Ana Silva	Alumno	Curso	10, 10, 10	Promedio: 10.0 -> APROBADO
V-202	Carlos Pérez	Alumno	Diplomado	14, 14, 13	Promedio: 13.6 -> REPROBADO
V-303	María Gómez	Alumno	Bootcamp	20, 20, 13	Promedio: 17.6 -> REPROBADO (Regla: Ninguna nota < 14)
V-404	Luis Rojas	Alumno	Bootcamp	14, 15, 14	Promedio: 14.3 -> APROBADO
P-505	Dr. Pedro Infante	Profesor	Backend	N/A	N/A

PARTE 1: Casos de Prueba para el Estudiante (Auto-Evaluación)

Antes de subir tu código a GitHub, asegúrate de que tu programa supere estos 3 escenarios sin arrojar errores o cerrarse inesperadamente.

CP-01 (Happy Path): Cálculo Polimórfico Correcto

- **Acción:** Registra a los 4 alumnos y asígnales las notas de la tabla superior. Ve a la Opción 6 (Reporte General).
- **Resultado Esperado:** El estatus de cada alumno debe coincidir exactamente con la columna "Resultado Interno Esperado". Presta especial atención a **María Gómez (V-303)**: aunque su promedio es alto (17.6), el sistema debe aplazarla porque es del Bootcamp y tiene una nota de 13.

CP-02 (Funcionalidad LIFO): Botón de Pánico / Deshacer

- **Acción:** Busca a Ana Silva (V-101) para ingresarle una nueva nota, pero por error teclea un **45**. Inmediatamente ve al Menú Principal y selecciona la **Opción 4: Deshacer**. Luego vuelve a buscar a Ana y mírale el perfil.
- **Resultado Esperado:** El **45** debió desaparecer de su historial como si nunca hubiera existido. Si el sistema calcula el promedio incluyendo ese 45, tu estructura de Pila (Stack) está fallando.

CP-03 (Funcionalidad FIFO): Exportación de Cola de Certificados

- **Acción:** Ejecuta la **Opción 5: Generar Cola de Certificados**. Luego, ve a la carpeta de tu proyecto y abre el archivo **certificados_pendientes.txt**.
- **Resultado Esperado:** El archivo de texto debe existir y contener **solo a 2 personas**, estrictamente en este orden: Ana Silva de primera, y Luis Rojas de segundo. Carlos y María jamás deben aparecer en ese archivo.

PARTE 2: Casos de Prueba para el Profesor (Evaluación Técnica)

El jurado evaluador ejecutará estos escenarios para comprobar la robustez del código, el manejo de memoria y el uso real de las estructuras de datos exigidas.

EVAL-01: Prueba de Persistencia Crítica (Simulación de Apagón)

- **La Prueba:** El profesor abrirá el programa, registrará un nuevo alumno cualquiera (Ej. V-999), ingresará una nota y **cerrará el programa abruptamente** (cerrando la terminal o forzando la detención). Luego, volverá a ejecutar el programa e irá a la Opción 6.
- **Criterio de Éxito:** El alumno V-999 y su nota deben aparecer en pantalla. Si desaparecen, significa que el estudiante solo guardó los datos en memoria RAM (Arreglos/Listas) y nunca construyó el flujo de escritura hacia los archivos **.txt**. (*Penalización alta*).

EVAL-02: Inspección de Código - Polimorfismo Real vs Falso

- **La Prueba:** El profesor abrirá los archivos `.java` o `.cpp` y buscará cómo el estudiante programó la regla para aprobar o reprobar.
- **Criterio de Éxito:** El estudiante debió crear un método `evaluarAprobacion()` en la clase padre `ProgramaAcademico`, y luego usar `@Override` (en Java) para sobrescribir ese método dentro de las clases hijas `Curso`, `Diplomado` y `Bootcamp`.
- **Bandera Roja:** Si el profesor encuentra un código en el archivo principal (`main`) lleno de condicionales como `if (tipo == "Bootcamp") { ... } else if (tipo == "Curso") { ... }`, significa que el alumno **no aplicó polimorfismo** y resolvió el problema usando programación estructurada básica. (*Penalización media*).

EVAL-03: Resiliencia del Sistema (Negative Testing - Try/Catch)

- **La Prueba:** En el Menú Principal, cuando el sistema pregunte *"Seleccione una opción (1-7):"*, el profesor escribirá una letra (Ej. `A`) o dejará el espacio en blanco y presionará Enter. También intentará registrar una nota escribiendo la palabra `veinte`.
- **Criterio de Éxito:** El programa no debe "romperse", congelarse, ni mostrar el código de error de la consola (Ej. `InputMismatchException` en Java). Debe capturar la excepción (`try-catch`), mostrar un mensaje amable en consola: *"Error: Ingrese un valor numérico válido"* y volver a mostrar el menú para que el usuario lo intente de nuevo.

EVAL-04: Auditoría de Memoria y Punteros (Exclusivo para C++)

- **La Prueba:** En el Módulo 6 (C++), el profesor revisará el código fuente buscando las palabras reservadas `new` y `delete`.
- **Criterio de Éxito:** Todo objeto creado dinámicamente con `new` (por ejemplo, al registrar un nuevo Alumno en la memoria) debe tener su correspondiente liberación de memoria usando `delete` antes de que el programa termine o al vaciar una estructura de datos. Si solo hay `new` y no hay `delete`, el estudiante creó "Fugas de Memoria" (Memory Leaks).